Cadence — 15-minute, three-person review

Order: You → Harsh → Aarush. **One laptop, one browser session.** Each person speaks for roughly five minutes and leaves the next person's chapter open.

The single story

> A sender hides 0s and 1s in the pauses between ordinary ping requests. We first inspect those pauses, then test whether a defender can spot or disrupt the message without harming normal traffic. Finally, we encrypt the note and ask what the defender can still learn. The site makes those ideas visible and gives us a workbench to test them; the included network runs are simulated.

Keep coming back to **three questions**: Can the receiver read the bits? Can the guardian notice the timing? What does the defence cost ordinary traffic?

The run of show

Clock	Speaker	Live route and purpose	Leave the laptop here
0:00–5:00	You	Guided case chapters 1–2, **Runs**, **Signal**. Explain how bits become waits, show where recordings come from, then investigate a suspicious trace.	Guided case, **Guardian** chapter open.
5:00–10:00	Harsh	Guided case **Guardian**, then **Detectors** and **Stress matrix**. Compare detection with false alarms; show the pacing trade-off and wider results grid.	Guided case, **Locked note** chapter open.
10:00–15:00	Aarush	Guided case **Locked note**, **Decode**, **Guide**, **Case closed**, **Evidence**. Demonstrate real browser encryption, decoding, limits, architecture, and the next experiment.	Evidence page and final claim visible.

This touches every main function on the top navigation. The **Analysis overview** in the footer is a map of the five workbench tools; mention it if someone asks where to begin, but it needs no separate live stop.

Exact handoffs

1. **You → Harsh:** Click **Continue: Guardian**. Say: "We can see a suspicious timing pattern. Harsh will show whether a rule can detect it reliably, and what happens to ordinary traffic." Hand him the laptop.
2. **Harsh → Aarush:** Return to **Guided case**, open **Locked note** in the chapter bar. Say: "A guardian can see timing and can even disrupt it. Aarush will show what changes when the note itself is encrypted." Hand him the laptop.
3. **Aarush → panel:** Finish on **Evidence**. Say: "Our software and browser encryption work, and the simulations show the trade-offs. The next claim needs real receiver-side captures; the page states that limit openly."

Keep these claims precise

- **610 included runs are simulated; zero included receiver captures are real.** The CSV importer accepts a user's capture, but importing a file does not prove how it was collected.
- The guided case computes from the included dataset. A high detection percentage is a result for a **small controlled set of windows**, not a universal accuracy guarantee.
- The paced relay is a **causal browser model** that holds packets; it has not been deployed on a router.

- AES-256-GCM encryption is performed by the browser's Web Crypto API. The encrypted bytes' **timing delivery is simulated**, with no ICMP packets sent by that scene.
- Our contribution is the integrated investigation and evaluation workflow: sender idea → receiver evidence → detector errors → defence cost → encrypted content → evidence limits. We are not claiming to have invented AES or covert timing channels.

Five-minute preflight on the review laptop

1. Keep both `C:\Users\adita\Desktop\CADENCE-V2` and the original `CNPROJECT` folder present: the local copy shares installed UI dependencies with the original to save disk space.
2. Open http://127.0.0.1:5174/#/case. If it does not load, open PowerShell and run:

`powershell cd C:\Users\adita\Desktop\CADENCE-V2\ui npm run dev -- --host 127.0.0.1 --port 5174 --strictPort`

Leave that PowerShell window open. Do not open `index.html` directly as a file. 3. Confirm the banner says the included recordings are simulated, and the first page says **610 simulated recordings** after loading. 4. Check that **Guardian** opens and **Locked note** → **Lock this note** succeeds. Reload the start URL afterward. 5. Keep this run sheet and the three speaker cards open on a phone or printed. Leave the laptop browser for the demo itself. Use a visible timer; rehearse the *clicks* as well as the words.

Recovery links if the browser lands on the wrong page

- Start / idea
- Detective
- Guardian
- Locked note
- Case closed
- Evidence

The top navigation reaches Runs, Signal, Detectors, Stress matrix, Decode, Guide, and Evidence. If a click or animation is slow, **say the point while it loads**. Skip an optional control before stealing time from the next presenter.

What to rehearse together

Run the whole sequence twice. The first pass is slow, with questions; the second pass uses a timer and the same laptop. Each person must practice **arriving at their starting screen**, speaking their first line, making their clicks, and delivering their handoff. Do a third pass only for the part that ran over time. Read common questions once as a team.

Speaker cards: You · Harsh · Aarush.

Presenter 1 — You: sender and detective (0:00–5:00)

Your question: How can a message hide in ping timing, and what evidence can we inspect? You own **The idea**, **Runs**, **Signal**, and **Detective**. Harsh owns the decision rules, so stop at the question "Is this suspicious enough to alarm?"

Start screen: Guided case — The idea. **End screen:** Guided case — Guardian. Leave the mouse with Harsh.

Your five-minute route

0:00–0:45 — Give the audience a reason to care

Point at the concept diagram. Say:

> "This project studies a hidden communication method. A ping is a simple network check. Its requests normally arrive about one second apart. A sender can keep the packets looking ordinary but change the *waiting time* between them. A short wait can stand for zero; a long wait can stand for one. We ask whether someone watching the receiver can spot that pattern."

Do not say the site is sending packets live. This first graphic is a concept diagram.

0:45–1:30 — Make the bits concrete

Scroll to "Turn a note into packet timing." Leave `HI` in **Your short note**, or type it again. Point at **16 bits** and the blue/orange squares.

> "Computers store text as bits, meaning zeros and ones. `HI` is two bytes, or sixteen bits. In this simple example, zero waits 0.75 seconds and one waits 1.25 seconds. The later investigation uses a subtler pair, 0.95 and 1.05 seconds, because a real question is whether the difference stays visible once the network adds delay."

One fact to remember: one byte = eight bits. A millisecond (ms) is one-thousandth of a second.

1:30–2:15 — Show what data the project has

Click `Runs` in the top navigation. Point to the source banner, the **610** loaded recordings, the **0** real captures, the CSV import area, and one row in the table. Do not import a file during the timed demo.

> "This is the recording table. It includes ordinary and message-carrying examples under different conditions. We can import receiver capture CSVs here and inspect their origin. Today's included 610 runs are simulated and there are no included real receiver captures. That source label matters whenever we quote a result."

If someone asks why 610: these are prepared examples across channels, conditions, seeds, and roles; **the count is not 610 independent real experiments**.

2:15–3:00 — Show the original analysis workbench

Click `Signal`. Point at **Choose runs**, the four measurement tiles, and the **Inter-packet delay over time** graph. Scroll a little to reveal **Gap distribution** if the projector allows it.

> "The detector works with arrival times at the receiver. We subtract consecutive arrival times to get each gap. The Signal view compares two recordings, measures their average and variation, and shows the gap distribution. Blue is ordinary traffic; orange is the message example. This is the detailed workbench behind the simpler detective scene."

Do not explain KS distance unless asked. It measures how different two distributions are; 0 is similar, 1 is far apart.

3:00–4:45 — Be the detective

Click `Guided case`, then chapter `02 Detective`. Keep the defaults: **Variation of 50 ms**, **0.95/1.05 s · subtle**, **Case file A**. Point at the blue reference and orange case, then the three measurements under the graph. Click **A hidden message** to reveal the labelled answer.

> "Here is the same idea as a small investigation. The blue trace is an ordinary reference and the orange trace is the unknown case. Each point is a pause measured in milliseconds. The case's pauses vary more, but one squiggly line is only a clue. The measurements below make the comparison repeatable. I guess this one carries a hidden message; the label confirms it because this run was generated by the simulator."

If time allows, select **Case file B** and guess **Ordinary pings**, then return to **Case file A** before the handoff. Skip this if the clock is past 4:20.

4:45–5:00 — Hand off cleanly

Click `Continue: Guardian`. Say:

> "We have a clue in the pauses. Harsh will now turn that clue into an alarm rule, count its mistakes, and test whether a defence can interfere with the message."

Stop speaking and hand him the laptop.

Your three memory anchors

1. **Packet contents can look normal while timing carries bits.**
2. **The receiver measures gaps; network variation makes them noisy.**
3. **The included data is simulated, so a graph is evidence about the model.**

If you get a question

- **"What is ICMP?"** The protocol used by ping. We look at the arrival times of its Echo Requests.
- **"Why measure at the receiver?"** The network can alter delays. The receiver sees what actually arrived.
- **"Is orange proof of an attack?"** No. In this controlled run we know its label; a real case needs repeated measurements and a detector with false alarms checked.
- **"What if a packet is lost?"** The analysis uses ICMP sequence numbers to avoid treating a missing request as one ordinary gap. Harsh and Aarush will discuss detection and recovery effects.

If the clock is running out: omit the Gap distribution scroll and Case file B. Preserve the final detective guess and the handoff. The Runs and Signal tabs should still be briefly visible.

Presenter 2 — Harsh: guardian and defence (5:00–10:00)

Your question: Can we detect or disrupt the hidden timing *without* accusing ordinary traffic or delaying it too much? You own **Guardian**, **Detectors**, and **Stress matrix**. You do not need to re-teach the sender's 0/1 encoding; the first speaker has done that.

Start screen: Guided case — Guardian. **End screen:** Guided case — Locked note. Leave the mouse with Aarush.

Your five-minute route

5:00–5:45 — Explain what a guardian actually judges

Point to the three rule choices. Say:

> "We should not declare an attack from one unusual pause. This guardian judges a small group of 32 requests. It measures how much their gaps vary, then compares that with a limit. We test three ways to choose the limit: one fixed number, a limit learned from ordinary traffic under the same conditions, and a limit learned only on a clean network."

Plain terms: a *window* is one group of requests; a *baseline* is a normal reference used to choose a limit; *standard deviation* means how much the pauses wobble around their average.

5:45–6:45 — Compare the rules, including mistakes

Begin on `Same conditions`. Point to **Messages caught** and **False alarms** together. On the default example they show **10/10 caught** and **3/10 ordinary windows falsely flagged**. Click **Fixed limit**: it usually shows **0/10 caught, 0/10 false alarms** on this default example. Click **Clean-only baseline**: it shows **10/10 caught but 10/10 false alarms**. Return to **Same conditions**.

> "If a rule never sounds the alarm, its false-alarm rate looks perfect but it misses the message. If it alarms on everything, it catches the message but becomes useless for ordinary users. The same-conditions baseline is more sensible here, yet even it still wrongly flags some normal traffic. These figures describe this small simulated set, not an accuracy guarantee for the Internet."

Read the live figures if the condition or dataset was changed. Do not memorize a percentage without its numerator and denominator.

6:45–7:20 — Make the statistics visible

Point to `Verdict by observation window`. Red squares in the ordinary row are false alarms; red squares in the message row are detections.

> "These squares are the actual decisions behind those percentages. The ordinary row shows the cost of mistakes; the message row shows what was caught. That is why we report both numbers together."

7:20–8:05 — Show the second defence and its cost

Point to `Try a second defence: pace the packets`. At the default **500 ms** waiting room, the screen shows about **90% bits read before**, **57% after**, and **+470 ms** added wait to ordinary pings. Move the slider briefly to **0**

ms: no added wait and no disruption. Return to **500 ms**.

> "This model releases requests on steadier one-second slots when it can. It cannot send a packet before it arrives; it only holds it. More waiting can damage the hidden timing, but it also delays innocent pings. The relay is a browser model, not a deployed router."

8:05–9:10 — Show the full detector workbench

Click `Detectors` in the top navigation. Point to the settings, the fixed-rule and baseline result tiles, and scroll once to the verdict chart or ROC/feature comparison. No need to tune sliders live.

> "This is the full instrument panel behind the guided guardian. We can choose data and network condition, window size, baseline percentile, fixed thresholds, and features. It calculates detection and false alarms, then gives per-window verdicts and comparison curves. The guided scene uses a narrow, readable subset of this same analysis."

If someone asks about the ROC curve: it plots detection against false alarms as a threshold changes. It does **not** turn this small simulated dataset into real-world validation.

9:10–9:45 — Show the broader test grid

Click `Stress matrix`. Point across the channel settings (rows) and network conditions (columns). Optionally change **Metric** from `detection rate` to `false-positive rate` if you have at least 20 seconds.

> "One attractive result can hide a fragile method. This grid asks what happens when we make the timing subtler or add delay and loss. We can switch the metric to see detection, false alarms, or decoding accuracy. This is where we look for failure cases, not just the best cell."

9:45–10:00 — Hand off cleanly

Click `Guided case`, then chapter `04 Locked note`. Say:

> "The guardian can see and disrupt timing, but it has not read the note itself. Aarush will show what encryption changes, what it costs, and what our evidence supports."

Stop speaking and hand him the laptop.

Your three memory anchors

1. **Detection rate and false-alarm rate must be read together.**
2. **A normal baseline should match the conditions it will face.**
3. **A pacing defence has a legitimate-traffic delay cost.**

If you get a question

- **"What exactly is the learned limit?"** The guided baseline uses the 95th-percentile standard deviation from separate ordinary 32-request windows; it alarms when a test window exceeds it.
- **"Why 32?"** It is the guided case's observation size, a trade-off between reacting quickly and collecting enough gaps. The full Detectors tool lets us change it; we do not claim 32 is universally optimal.
- **"Is 100% detection a proven result?"** No: here it means 10 of 10 message windows in one controlled simulated condition. Also state the false alarms and the absence of real captures.

- **"Did you install this relay on a router?"** No. The algorithm is a causal model in the browser. A real deployment would need latency and throughput measurements on lab hardware.

If the clock is running out: show one comparison in Guardian, the pacing cost, then briefly expose the Detectors and Stress matrix pages. Keep the 10:00 handoff; Aarush needs his full five minutes.

Presenter 3 — Aarush: encryption, decoding, and evidence (10:00–15:00)

Your question: If the note is encrypted, what can the receiver and guardian still do—and what have we really proved? You own **Locked note**, **Decode**, **Guide**, **Case closed**, and **Evidence**. Do not repeat the detector-rule comparison; Harsh has covered it.

Start screen: Guided case — Locked note. **End screen:** Evidence page, with the limits and next experiment stated aloud.

Your five-minute route

10:00–10:35 — Set up the new question

Point to the four-stage flow: note → lock → timing bits → receiver. Say:

> "Harsh showed that the guardian can notice timing. Now we ask a different question: can it read the message? We lock the text using AES-GCM in the browser, then model those encrypted bytes as short and long waits. The intended receiver knows the key phrase; the guardian sees the pauses, not the phrase."

Plain distinction: encryption hides *contents*. It does not automatically make a timing pattern look ordinary.

10:35–11:55 — Run the locked-note experiment

Leave `HI` and the prefilled demo key phrase. Click `Lock this note`. Point to **2 bytes → 46 bytes → 368 bits → 6 min 8 sec**. Point to **timing bits recovered**, the successful **HI** unlock, and the guardian result. Click **Try a wrong key**; it should say **Could not unlock**. If time permits, restore the receiver phrase by typing `cadence-demo-key`, then set **Network delay variation** to **50 ms** to show that damaged encrypted bits can also block the unlock.

> "The encryption is real browser cryptography. Even a two-letter note becomes a 46-byte package because it needs a salt, one-time value, and integrity check. At roughly one timing bit per second, that is over six minutes of modelled transmission. With the right key and intact bits, the receiver gets `HI`. A wrong key fails. The guardian may still flag the timing without knowing the note. Encryption changes secrecy, not the visibility or cost of this channel."

Read the guardian's flagged-chunk count from the screen; it can change when fresh random encryption bytes are made. **No actual ping is sent by this experiment.** The prefilled phrase is for demonstration, not a strong production secret.

11:55–12:35 — Show the original decoder

Click `Decode` in the top navigation. Point to **Try your own message**, **Choose a covert run**, the threshold choices, the recovered-bit result, and the sent/decoded bit strip. You can leave its defaults; the upper ideal example and the prepared noisy run intentionally have different outcomes.

> "Decode is the detailed receiver view. It turns a received short or long gap back into zero or one. The top toy example is clean and should recover the note. Below, a prepared run includes network variation, so we compare sent and recovered bits and try fixed or adaptive dividing lines. A high bit percentage is useful, but an encrypted message needs *every* bit intact to unlock."

Important: the guided locked note and this prepared decoder run are related demonstrations, not the same live packet trace.

12:35–12:55 — Show that the site teaches its vocabulary

Click `Guide`. Type `jitter` into **Find a word** and point to the explanation. Clear search only if you have time.

> "The guide explains the units, charts, and terms inside the product. Jitter simply means packet delay changes from one request to the next."

12:55–13:20 — Close the guided story

Click `Guided case`, then chapter `05 Case closed`. Point to the three summary cards.

> "The full story is sender, detective, guardian, and encrypted receiver. A message can be readable, detectable, both, or neither; a defence also has a cost. The final question is what our evidence actually supports."

Click `Inspect evidence and limits`.

13:20–14:35 — Make the evidence boundary explicit

On `Evidence`, point to the three counts and the three levels of evidence. Show **610 simulated recordings**, **0 real-capture labels**, and **0 unverified imports** on a fresh session. Point to **Turn the case into a measured result**. If time permits, scroll to the export buttons; do not start a download unless asked.

> "The software can read runs, calculate gaps, compare detectors, model pacing, and encrypt and unlock in the browser. The shipped examples are simulations. We have not included real receiver-side lab captures, so we do not claim measured real-network detection performance. Our next experiment is to capture ordinary and encoded pings at the receiver, repeat under delay and loss, and report detection, false alarms, recovered bits, and added delay."

14:35–15:00 — Give the architecture and final claim

> "Under the hood, Python tools generate or capture runs and export a versioned JSON bundle. The React/TypeScript browser reads that bundle, computes the analysis, draws the charts, and uses Web Crypto for the locked note. Our contribution is a coherent, inspectable attacker–defender learning and evaluation workflow. Real lab captures are the next validation step."

Stop there. Leave **Evidence** visible for questions.

Your three memory anchors

1. **Real encryption; modelled packet timing.**
2. **Correct key plus intact bits are required to recover the note.**
3. **Working software and simulated results are distinct from real-network proof.**

If you get a question

- **“Did you invent AES?”** No. The browser uses standard AES-256-GCM. Our project connects encryption to the timing-channel investigation.
- **“Why does `HI` take six minutes?”** The encrypted package has 46 bytes, or 368 bits. At about one bit per second, that is roughly 368 seconds. It is an estimate from the model.
- **“Can the guardian decrypt it?”** It does not have the key. It can still examine the timing metadata and raise an alarm.
- **“What if one bit flips?”** AES-GCM's integrity check rejects the whole package; bit recovery and whole-message recovery are different measures.
- **“Where is the backend?”** The live UI is a browser app reading a prepared data bundle. Python scripts handle generating/exporting examples and the planned lab capture workflow; the browser does the interactive analysis and encryption.

If the clock is running out: keep Locked note and Evidence. Show Decode and Guide for 10–15 seconds each, then deliver the final honest claim. Do not consume extra time trying to make random encrypted bytes produce a particular flagged count.

Team question bank and architecture card

Read this together once. In the review, answer in **one plain sentence first**, then add detail only if the reviewer asks. If a question belongs to another segment, let that presenter answer it.

The system in one picture

```
Python simulator ──> versioned JSON example bundle ──> browser UI
Lab sender/capture scripts ──> receiver CSV import ─────> browser UI
                                                          │
                                                          ├─ gap and window measurements
                                                          ├─ fixed and learned detector rules
                                                          ├─ charts, tables, reports, and exports
                                                          └─ bit decoder

Browser Web Crypto ──> encrypted bytes ──> timing model ──> receiver unlock attempt
                                       └─> guardian timing check
```

The interface is React/TypeScript. The guided scenes reuse the analysis functions behind the detailed workbench. Data is fetched as a versioned JSON bundle and validated in the browser. CSV parsing runs in a browser worker. The browser uses Web Crypto for AES-256-GCM. Python also contains the simulator, detector reference implementation, tests, and the closed-lab send/capture scripts. The current guided demo does not send network packets.

Core questions

Reviewer asks	Short answer	Best speaker
What is ICMP?	The protocol used by ping; we examine the arrival times of its Echo Requests.	You
What is a covert timing channel?	A sender encodes information in *when* otherwise ordinary packets arrive, rather than in their visible contents.	You
How does zero or one get sent?	A short planned gap means 0, a long planned gap means 1; the receiver compares each received gap with a dividing line.	You / Aarush
Are you changing packet payloads?	The timing-channel idea changes gaps; packet-field equivalence still needs real lab captures and field checks to verify.	You
Why receiver-side timestamps?	The receiver observes the timing after network delay and loss have acted on it.	You
What is jitter?	Variation in network delay from packet to packet; it can blur the short/long difference.	You
What does “ms” mean?	Millisecond: one-thousandth of a second. The subtle demo gaps are 950 and 1050 ms.	You
Why compare a normal reference?	Without ordinary traffic under comparable conditions, an unusual-looking gap has no useful context.	You / Harsh
What is a window?	One group of consecutive requests judged together; the guided case uses 32 requests, giving up to 31 gaps.	Harsh

Reviewer asks	Short answer	Best speaker
What is standard deviation?	A measure of how much the gaps wobble around their average.	Harsh
How is the learned baseline made?	From separate ordinary runs, it learns a 95th-percentile limit for the chosen feature; the guided case tests standard deviation.	Harsh
What is the fixed rule?	In the guided case it alarms if standard deviation exceeds 100 ms or mean gap drifts over 100 ms from one second.	Harsh
Why did the fixed rule get 0% false alarms?	On this default condition it raised no alarms at all, so it also missed all ten message windows.	Harsh
Why can the clean-only baseline be bad?	A threshold learned in a clean network may treat ordinary noisy traffic as suspicious; the default example flags all ten ordinary windows.	Harsh
What does “100% detected” mean here?	Ten of ten message windows in this controlled simulated setting. It is not a universal accuracy claim.	Harsh
What is a false positive?	A normal window wrongly marked suspicious; it costs attention or causes unnecessary intervention.	Harsh
What do ROC and AUC mean?	ROC shows detection versus false alarms while the threshold changes; AUC summarizes that curve. These plots still use simulated data.	Harsh
What is the Stress matrix for?	It checks many combinations of channel subtlety, network variation, loss, detector choice, and metric so we can see failure cases.	Harsh
What does pacing do?	A model holds requests toward regular one-second release times. It may blur hidden timing, but adds delay for ordinary traffic.	Harsh
Is pacing deployed?	No. It is a causal browser model; a real relay needs implementation and measured network tests.	Harsh
What is the adaptive decoder?	It finds a dividing line between observed short and long gaps by clustering them into two groups; the fixed decoder uses one second.	Aarush
What happens if a request is lost?	Sequence numbers let the analysis omit gaps crossing a missing request. Missing bits still hurt recovery. The locked-note model itself adds jitter, not packet loss.	Aarush
Is the encryption real?	Yes. The browser uses Web Crypto AES-256-GCM, with a key derived from the phrase using PBKDF2 and a fresh random salt and IV. The timing delivery is modelled.	Aarush
Is the prefilled phrase secure?	It is a demo phrase. A real system would need a strong secret and a reviewed key-management design.	Aarush
Why does a two-letter note become 46 bytes?	Two text bytes plus 16 salt bytes, 12 IV bytes, and a 16-byte authentication tag.	Aarush
Why around six minutes?	46 bytes make 368 bits; this model uses about one gap per bit at roughly one second per gap.	Aarush
Does encryption hide the timing channel?	It hides note contents. The guardian can still observe abnormal pauses, and noise can still corrupt the encrypted bytes.	Aarush
Why does one wrong bit break decryption?	AES-GCM verifies the whole encrypted package; if the key or authenticated bytes are wrong, it rejects the message.	Aarush
Is there a backend/database?	The demo is a browser app loading a local JSON bundle. Python prepares data and offers lab tools; interactive analysis and encryption run in the browser.	Aarush

Reviewer asks	Short answer	Best speaker
What is the unique contribution?	One inspectable teaching/evaluation workflow joins encoding, receiver evidence, detection errors, defence cost, encrypted content, and provenance. We use established algorithms rather than claim a new cipher.	Aarush
What is your strongest honest result?	The software works on its controlled simulated examples and shows concrete trade-offs. Real-network performance remains unmeasured in the shipped bundle.	Aarush
What is next?	Collect repeated receiver-side ordinary and message runs on a closed lab testbed under clean/noisy/lossy conditions, separate training from testing, then report detection, false alarms, bit recovery, and added delay.	Aarush

Three answers the whole team must say the same way

1. **"Are these real lab measurements?"** "No. The shipped bundle has 610 simulated runs and zero included real receiver captures. The site labels them throughout."
2. **"Did you invent the cipher or detection statistics?"** "No. The originality is in integrating them into a transparent sender–detective–guardian evaluation and teaching tool, and exposing the trade-offs."
3. **"Is this production-ready network security?"** "No. It is a tested teaching and analysis tool. A real deployment needs capture data, field checks, traffic-cost measurements, and security review."

If the panel asks about tests or evidence quality

The local V2 passed 72 UI unit tests, 62 Python tests (2 skipped), a production build and size check, and 32 desktop/mobile browser checks (4 intentionally skipped). Browser checks include navigation, encryption success/failure, accessible controls, and layout widths. These checks support **software correctness**, not real-network accuracy. The evidence page includes report/data exports and clearly distinguishes software behaviour, simulation, and required real measurements.

Terms to avoid mixing up

- **Detection rate** = message windows flagged / all message windows.
- **False-positive rate** = ordinary windows wrongly flagged / all ordinary windows.
- **Bit recovery** = correct bits / expected bits; a high percentage can still leave the whole encrypted note unreadable.
- **Encryption** protects content; **encoding** maps bits into timing; **detection** judges observed timing; **pacing** delays release to reduce that timing pattern.
- **Simulated run** is generated data. **Real capture** requires actual receiver-side packet timestamps and a trustworthy record of how they were collected.